

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____
Преподаватель _____ *С. В. Умбетов*
« _____ » _____ 2026 г.

Отчёт по практической работе
Тема: «Табличные данные в
Pandas»
УП 09.02.08.03.001

Студент группы ИИИС-41 _____ *Р.Н. Анненко*
группа и.о., фамилия

Преподаватель доцент _____ *С. В. Умбетов*
должность, ученая степень и.о., фамилия

Цель работы

Освоить базовые и продвинутые приёмы работы с табличными данными в библиотеке Pandas на примере реального набора данных о пассажирах «Титаника». Научиться загружать, исследовать, фильтровать, агрегировать и преобразовывать табличные данные, а также делать аналитические выводы на основе полученных результатов.

Задачи к работе

В ходе выполнения лабораторной работы необходимо:

1. Изучить теоретические основы работы с библиотекой Pandas: объекты Series и DataFrame, их создание и основные свойства.
2. Освоить чтение данных из CSV-файлов с помощью функции `pd.read_csv()` и её параметров.
3. Научиться получать первичную информацию о датафрейме с помощью методов `.head()`, `.tail()`, `.info()`, `.describe()`.
4. Освоить индексацию и доступ к данным через `.loc`, `.iloc` и обращение к столбцам.
5. Научиться фильтровать данные по одному и нескольким условиям с использованием логических операторов.
6. Освоить статистические методы (`.mean()`, `.max()`, `.min()`, `.sum()`, `.std()`, `.count()`) и их применение к датафреймам и отдельным столбцам.
7. Изучить группировку данных с помощью `.groupby()` и агрегацию через `.agg()`.
8. Научиться строить сводные таблицы с помощью `.pivot_table()`.
9. Освоить работу с пропущенными значениями: обнаружение, удаление, заполнение.
10. Научиться создавать новые признаки на основе существующих с использованием арифметических операций, `pd.cut()` и `.apply()`.
11. Провести комплексный аналитический анализ датасета Titanic и сформулировать содержательные выводы.
12. Закрепить полученные навыки ответами на контрольные вопросы.

Задание принял: _____



подпись

Анненко Р.Н.

Задача 1. Загрузка и первичный осмотр данных.

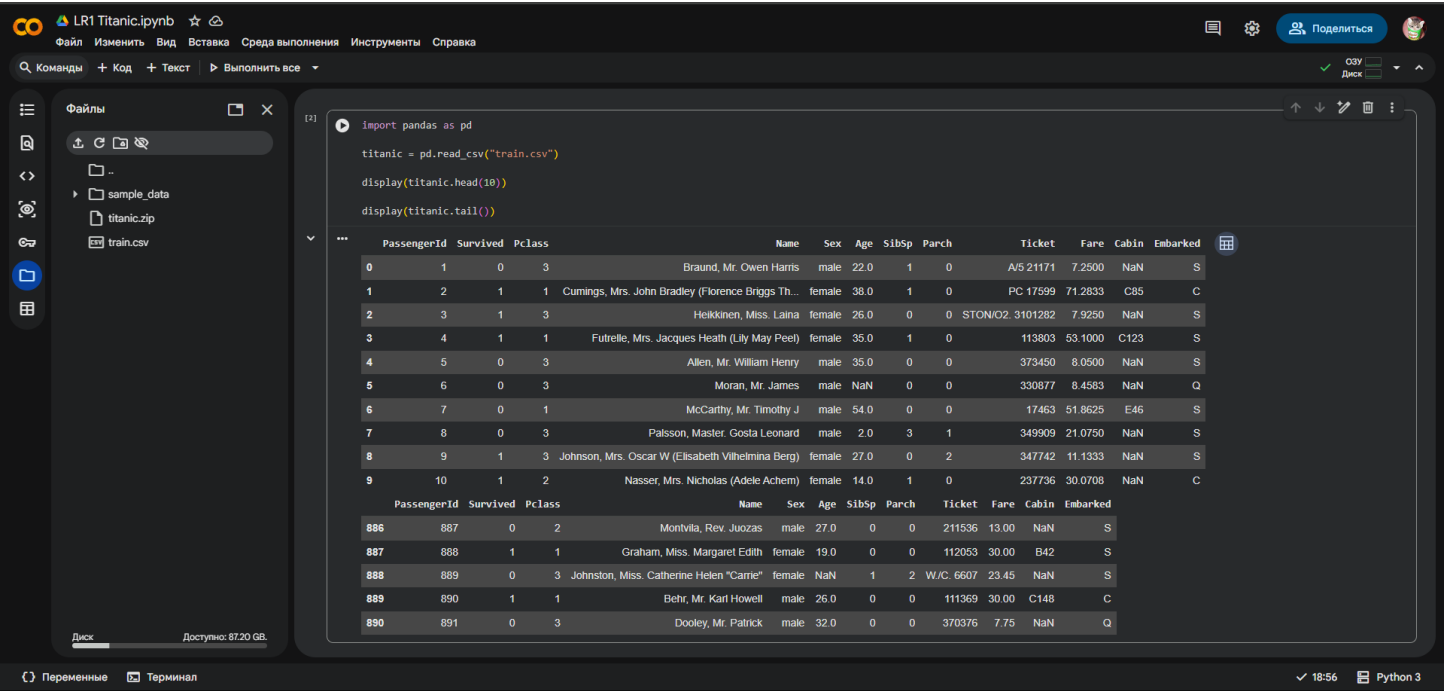


Рисунок 1 – Загрузил датасет, вывел первые 10 и последние 5 строк таблицы для проверки.

Задача 2. Получение информации о датафрейме.

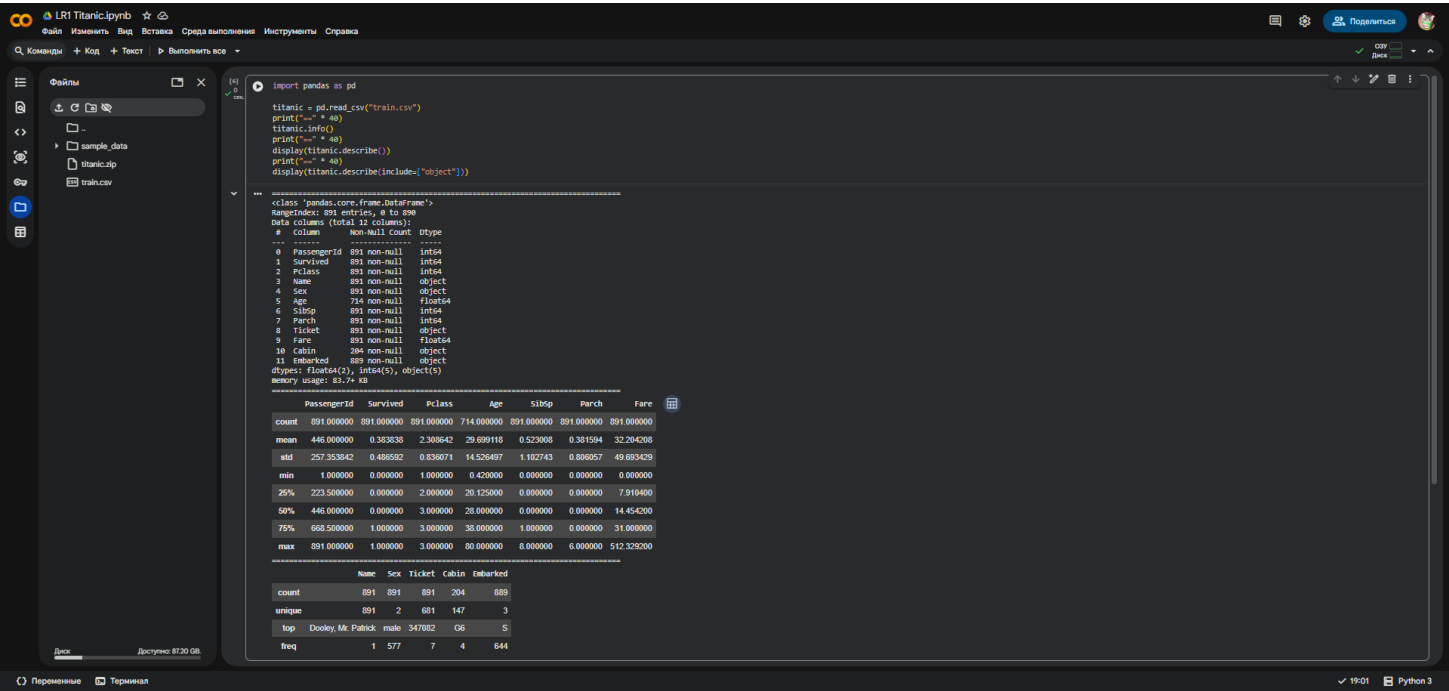
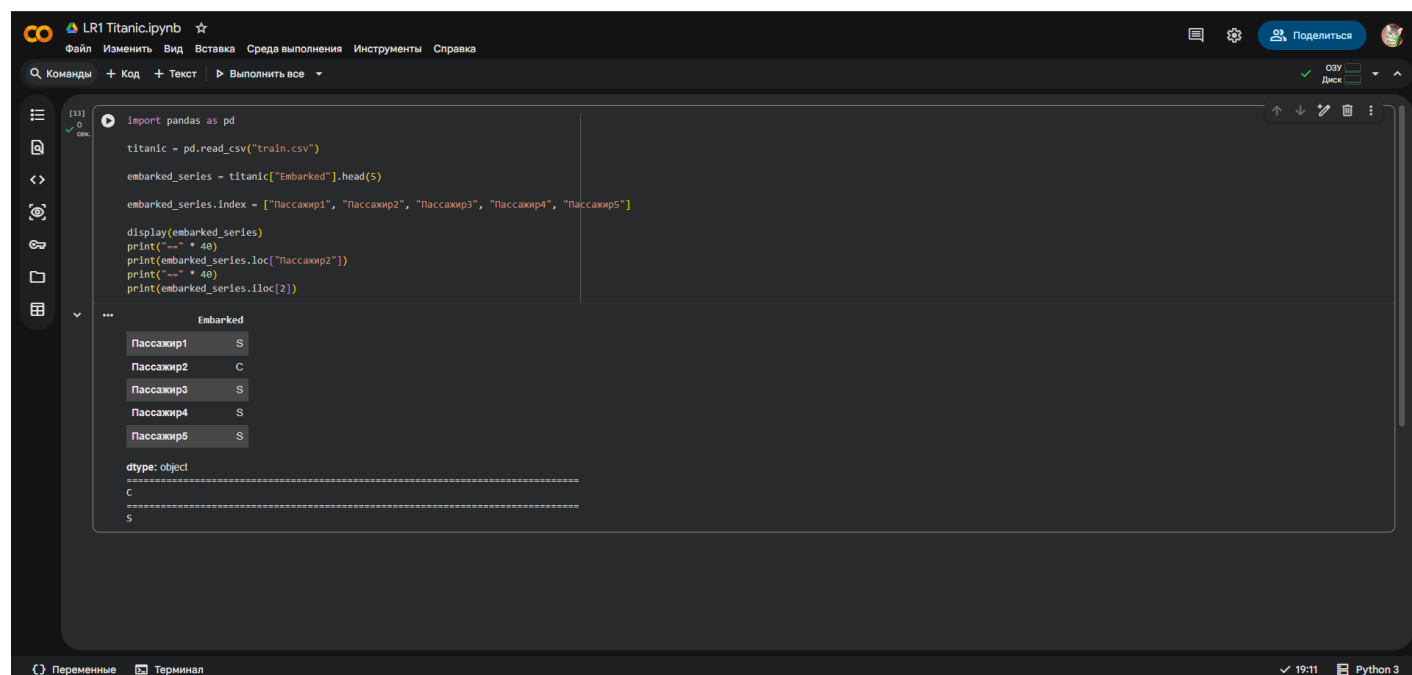


Рисунок 2 – Посмотрел общую информацию о типах данных.

Задача 3. Работа с Series.



```
[11]: import pandas as pd

titanic = pd.read_csv("train.csv")

embarked_series = titanic["Embarked"].head(5)

embarked_series.index = ["Пассажир1", "Пассажир2", "Пассажир3", "Пассажир4", "Пассажир5"]

display(embarked_series)
print("---" * 40)
print(embarked_series.loc["Пассажир2"])
print("---" * 40)
print(embarked_series.iloc[2])
```

Embarked	
Пассажир1	S
Пассажир2	C
Пассажир3	S
Пассажир4	S
Пассажир5	S

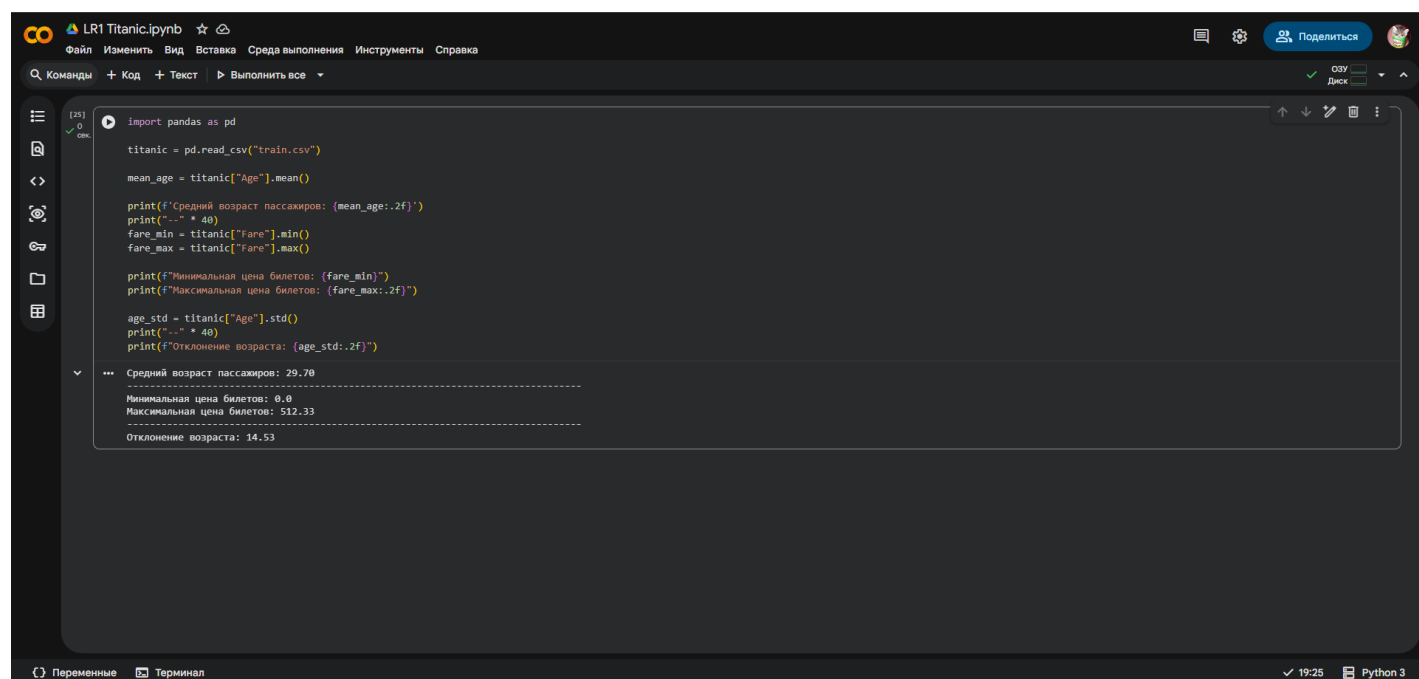
dtype: object

C

S

Рисунок 3 – Создал объект Series из столбца Embarked, поменял индексы на имена и проверил работу методов .loc и .iloc

Задача 4. Статистические методы.



```
[25]: import pandas as pd

titanic = pd.read_csv("train.csv")

mean_age = titanic["Age"].mean()

print(f"Средний возраст пассажиров: {mean_age:.2f}")
print("---" * 40)
fare_min = titanic["Fare"].min()
fare_max = titanic["Fare"].max()

print(f"Минимальная цена билетов: {fare_min}")
print(f"Максимальная цена билетов: {fare_max:.2f}")

age_std = titanic["Age"].std()
print("---" * 40)
print(f"Отклонение возраста: {age_std:.2f}")
```

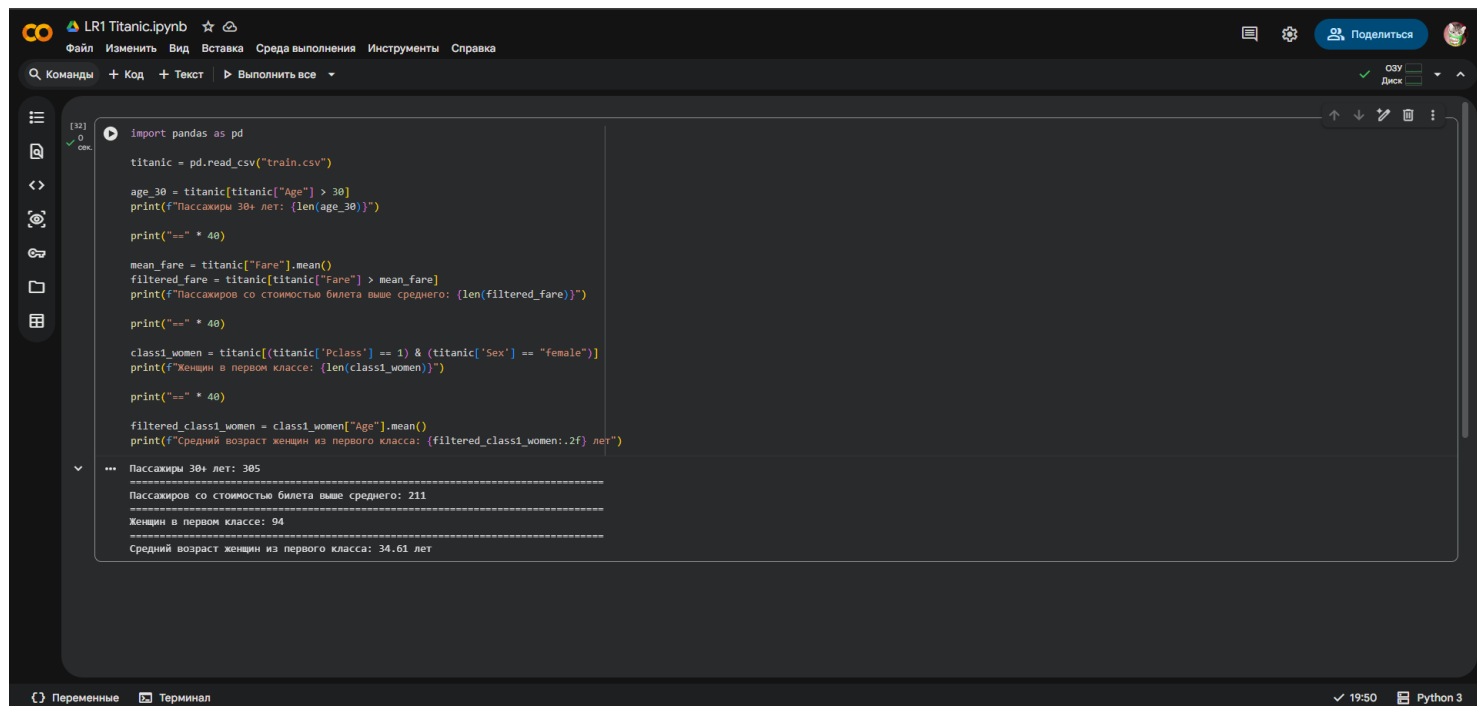
Средний возраст пассажиров: 29.70

Минимальная цена билетов: 0.0
Максимальная цена билетов: 512.33

Отклонение возраста: 14.53

Рисунок 4 – Посчитал средний возраст и цены билетов.

Задача 5. Фильтрация данных по условиям



```
import pandas as pd

titanic = pd.read_csv("train.csv")

age_30 = titanic[titanic["Age"] > 30]
print(f"Пассажиры 30+ лет: {len(age_30)}")

print("==" * 40)

mean_fare = titanic["Fare"].mean()
filtered_fare = titanic[titanic["Fare"] > mean_fare]
print(f"Пассажиров со стоимостью билета выше среднего: {len(filtered_fare)}")

print("==" * 40)

class1_women = titanic[(titanic["Pclass"] == 1) & (titanic["Sex"] == "female")]
print(f"Женщин в первом классе: {len(class1_women)}")

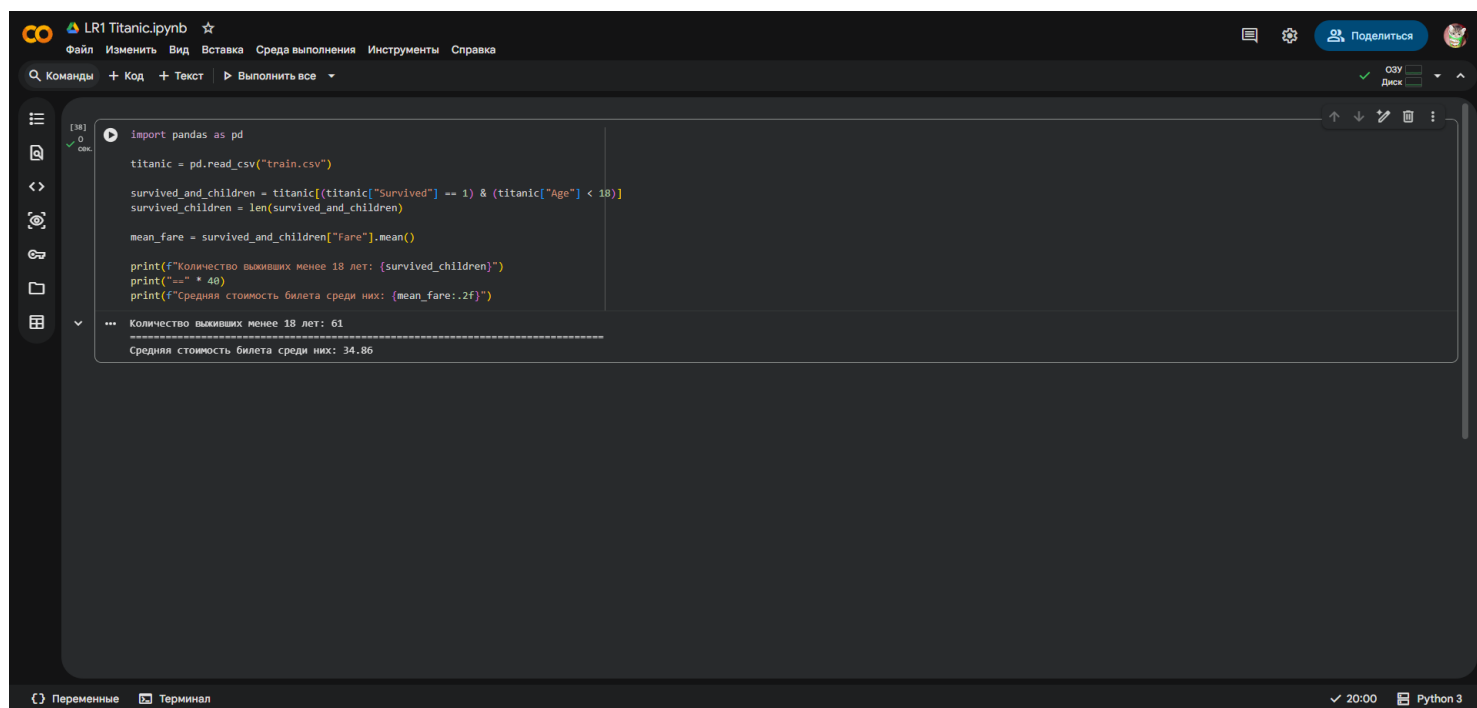
print("==" * 40)

filtered_class1_women = class1_women["Age"].mean()
print(f"Средний возраст женщин из первого класса: {filtered_class1_women:.2f} лет")

*** Пассажиры 30+ лет: 305
=====
Пассажиров со стоимостью билета выше среднего: 211
=====
Женщин в первом классе: 94
=====
Средний возраст женщин из первого класса: 34.61 лет
```

Рисунок 5 – Отфильтровал пассажиров старше 30 лет, билеты выше среднего и нашёл средний возраст женщин из 1-го класса.

Задача 6. Комбинированная фильтрация и статистика



```
import pandas as pd

titanic = pd.read_csv("train.csv")

survived_and_children = titanic[(titanic["Survived"] == 1) & (titanic["Age"] < 18)]
survived_and_children = len(survived_and_children)

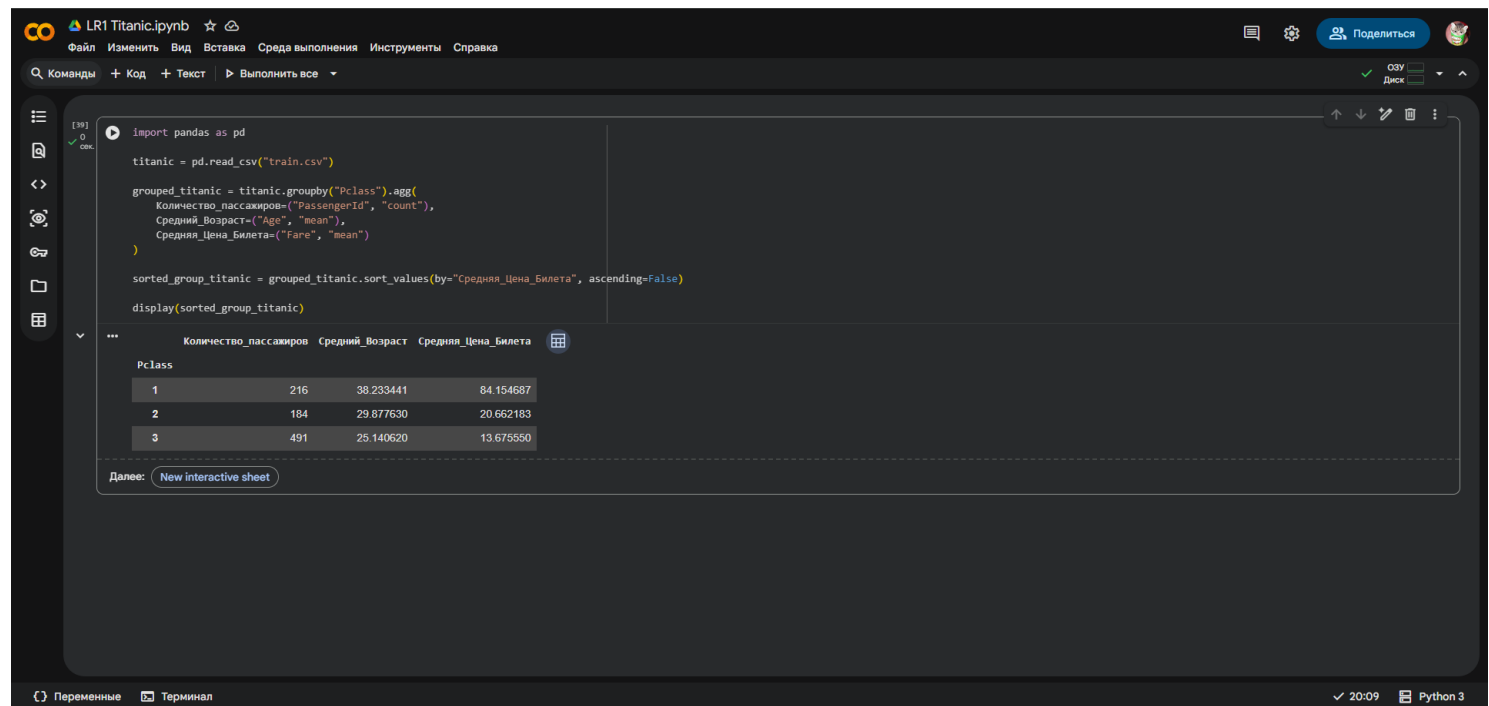
mean_fare = survived_and_children["Fare"].mean()

print(f"Количество выживших менее 18 лет: {survived_and_children}")
print("==" * 40)
print(f"Средняя стоимость билета среди них: {mean_fare:.2f}")

*** Количество выживших менее 18 лет: 61
=====
Средняя стоимость билета среди них: 34.86
```

Рисунок 6 – Нашёл всех выживших детей (моложе 18 лет) и посчитал среднюю цену их билетов.

Задача 7. Группировка данных



The screenshot shows a Jupyter Notebook interface with the following code and output:

```
import pandas as pd

titanic = pd.read_csv("train.csv")

grouped_titanic = titanic.groupby("Pclass").agg(
    количество_пассажиров=("PassengerId", "count"),
    средний_возраст=("Age", "mean"),
    средняя_цена_билета=("Fare", "mean")
)

sorted_group_titanic = grouped_titanic.sort_values(by="Средняя_Цена_Билета", ascending=False)

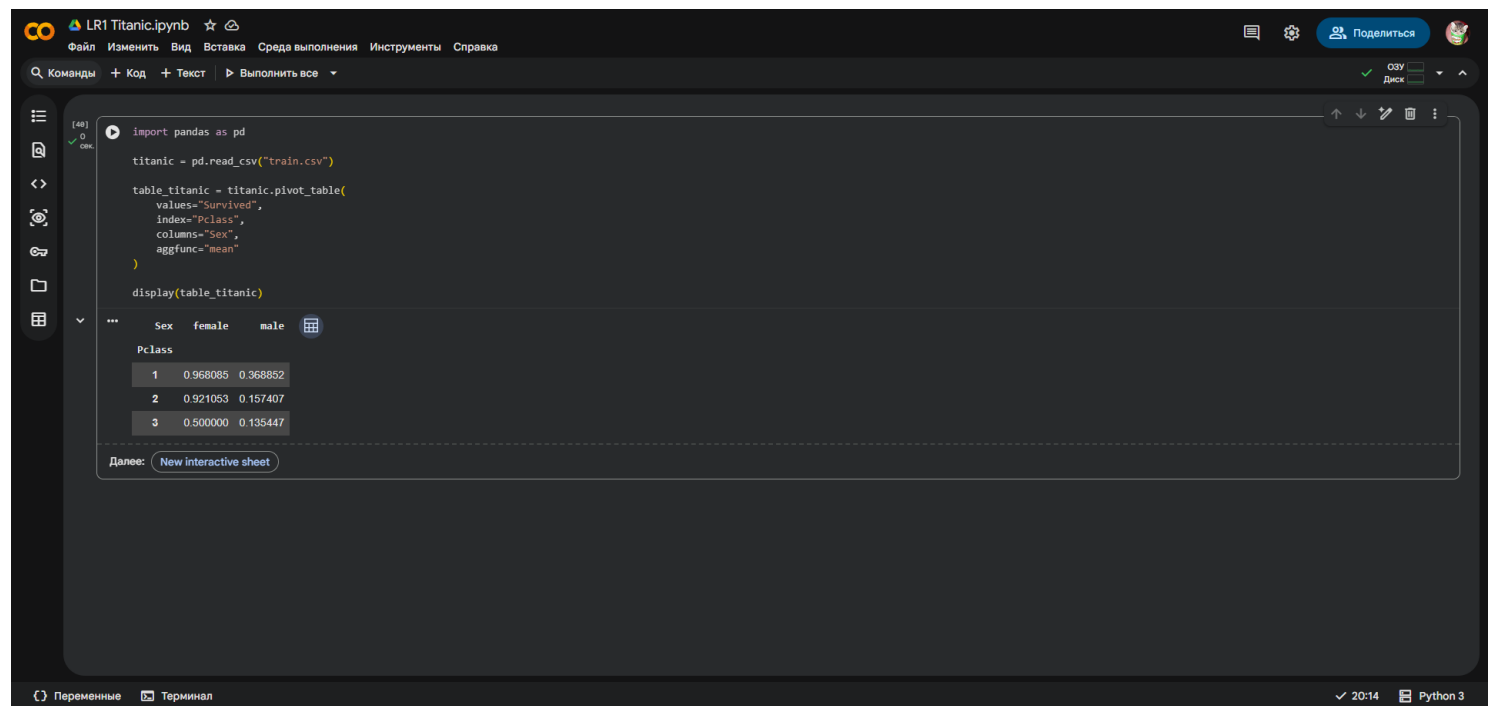
display(sorted_group_titanic)
```

The output is a table with the following data:

Pclass	количество_пассажиров	средний_возраст	Средняя_Цена_Билета
1	216	38.233441	84.154687
2	184	29.877630	20.662183
3	491	25.140620	13.675550

Рисунок 7 – Сгруппировал данные по классам билетов.

Задача 8. Сводная таблица



The screenshot shows a Jupyter Notebook interface with the following code and output:

```
import pandas as pd

titanic = pd.read_csv("train.csv")

table_titanic = titanic.pivot_table(
    values="Survived",
    index="Pclass",
    columns="Sex",
    aggfunc="mean"
)

display(table_titanic)
```

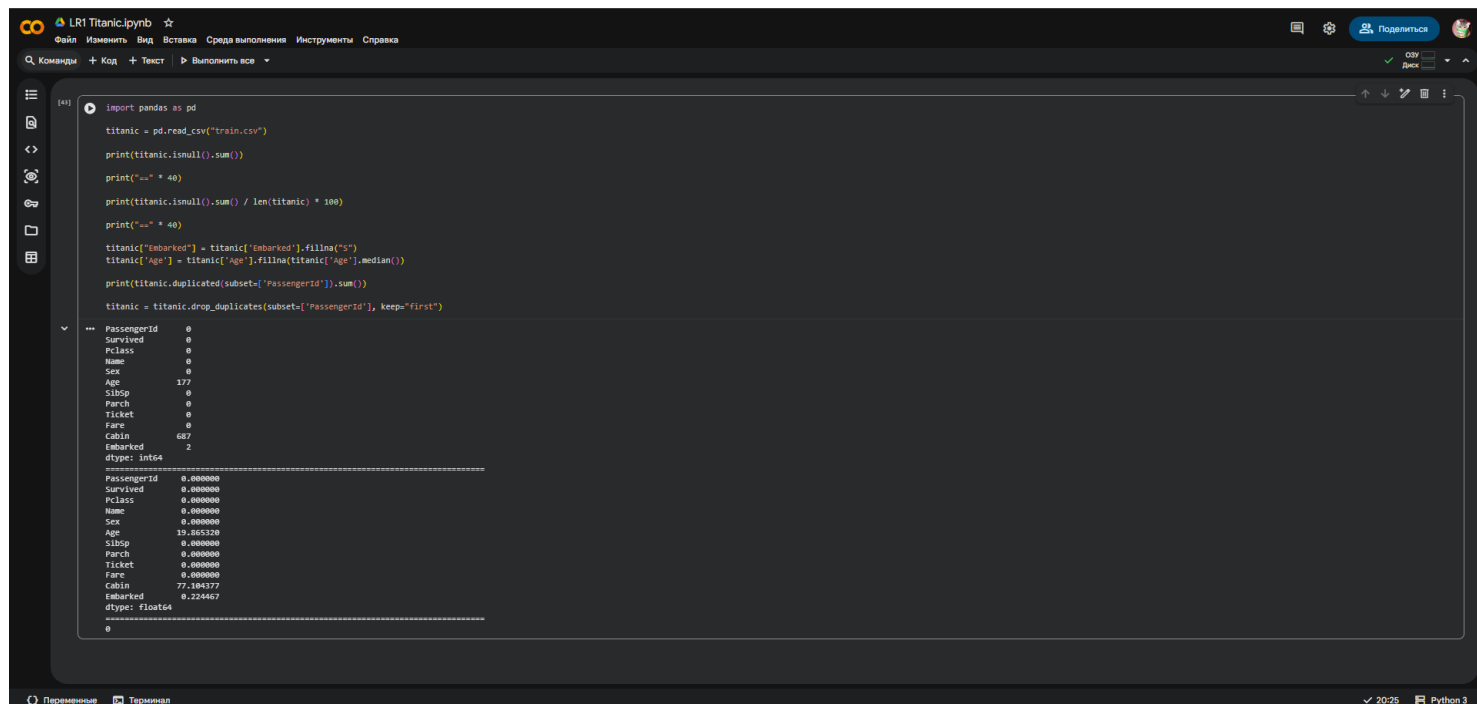
The output is a pivot table with the following data:

Pclass	Sex	female	male
1	female	0.968085	0.368852
2	female	0.921053	0.157407
3	female	0.500000	0.135447

Рисунок 8 – Построил сводную таблицу выживаемости.

Анализ: наибольшие шансы на спасение имели женщины из первого класса (около 97%), а наименьшие у мужчин из третьего класса (всего 13.5%).

Задача 9. Работа с пропущенными значениями



```
[40]: import pandas as pd

titanic = pd.read_csv("train.csv")

print(titanic.isnull().sum())

print("\n" * 40)

print(titanic.isnull().sum() / len(titanic) * 100)

print("\n" * 40)

titanic["Embarked"] = titanic["Embarked"].fillna("S")
titanic["Age"] = titanic["Age"].fillna(titanic["Age"].median())
print(titanic.duplicated(subset=["PassengerId"]).sum())

titanic = titanic.drop_duplicates(subset=["PassengerId"], keep="first")

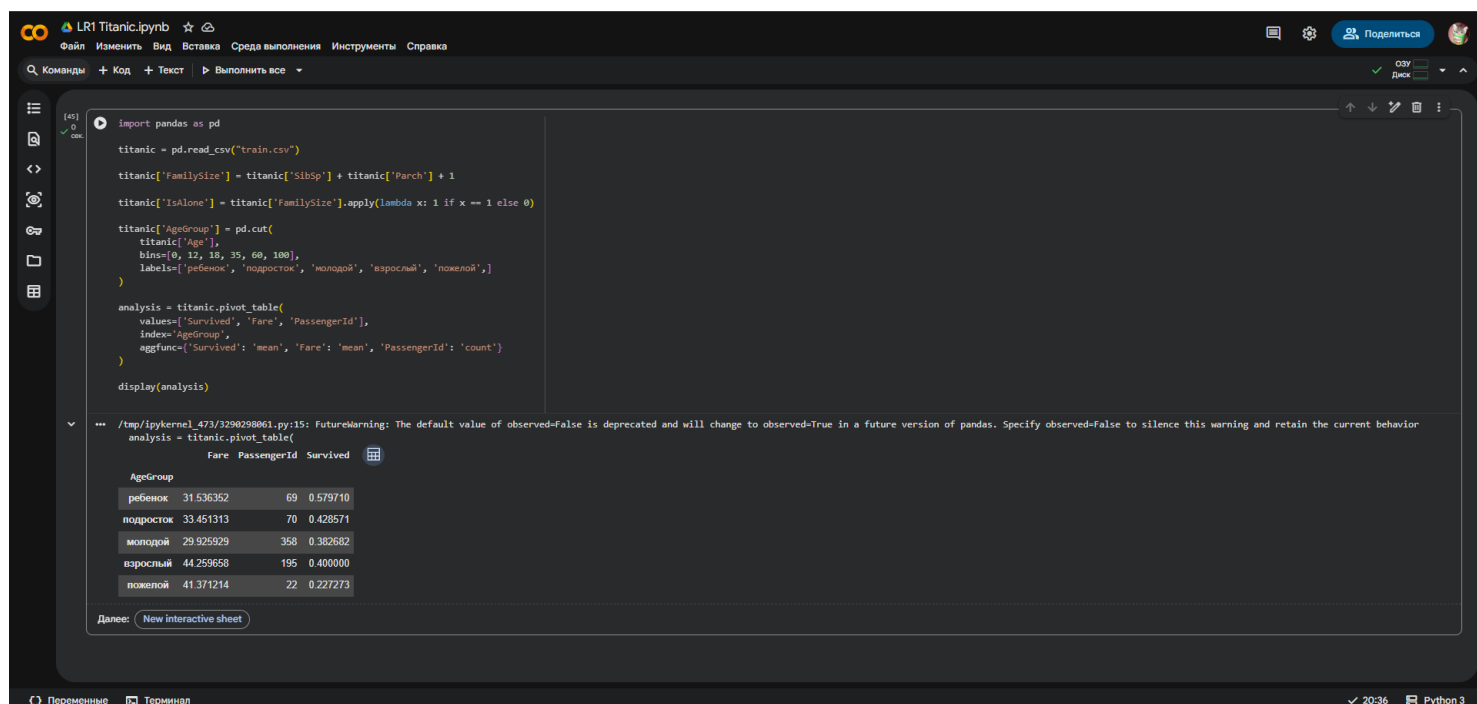
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch         0
Ticket         0
Fare           0
Cabin        687
Embarked       2
dtype: int64

-----
PassengerId    0.000000
Survived       0.000000
Pclass        0.000000
Name          0.000000
Sex           0.000000
Age          19.865328
SibSp         0.000000
Parch         0.000000
Ticket        0.000000
Fare          0.000000
Cabin        77.194377
Embarked      0.224467
dtype: float64

0
```

Рисунок 9 – нашел пропуски в процентах. Заполнил пустой возраст медианой, порты – буквой 'S', дубликатов в таблице не оказалось.

Задача 10. Создание новых признаков и расширенный анализ



```
[40]: import pandas as pd

titanic = pd.read_csv("train.csv")

titanic["FamilySize"] = titanic["SibSp"] + titanic["Parch"] + 1
titanic["IsAlone"] = titanic["FamilySize"].apply(lambda x: 1 if x == 1 else 0)

titanic["AgeGroup"] = pd.cut(
    titanic["Age"],
    bins=[0, 12, 18, 35, 60, 100],
    labels=["ребенок", "подросток", "молодой", "взрослый", "пожилой"],
)

analysis = titanic.pivot_table(
    values=["Survived", "Fare", "PassengerId"],
    index="AgeGroup",
    aggfunc={"Survived": "mean", "Fare": "mean", "PassengerId": "count"}
)

display(analysis)

analysis = titanic.pivot_table(
    values=["Survived", "Fare", "PassengerId"],
    index="AgeGroup",
    aggfunc={"Survived": "mean", "Fare": "mean", "PassengerId": "count"}
)

display(analysis)
```

AgeGroup	Survived	Fare	PassengerId
ребенок	0.579710	31.536352	69
подросток	0.428571	33.451313	70
молодой	0.382682	29.925929	358
взрослый	0.400000	44.259658	195
пожилой	0.227273	41.371214	22

Рисунок 10 – Создал новые колонки (размер семьи и одиночки). Разбил людей на возрастные группы

Вывод: Самая высокая доля выживших зафиксирована в группе «ребёнок» (58%), а самая низкая – в группе «пожилой» (22%).

Задача 11. Комплексный аналитический отчёт

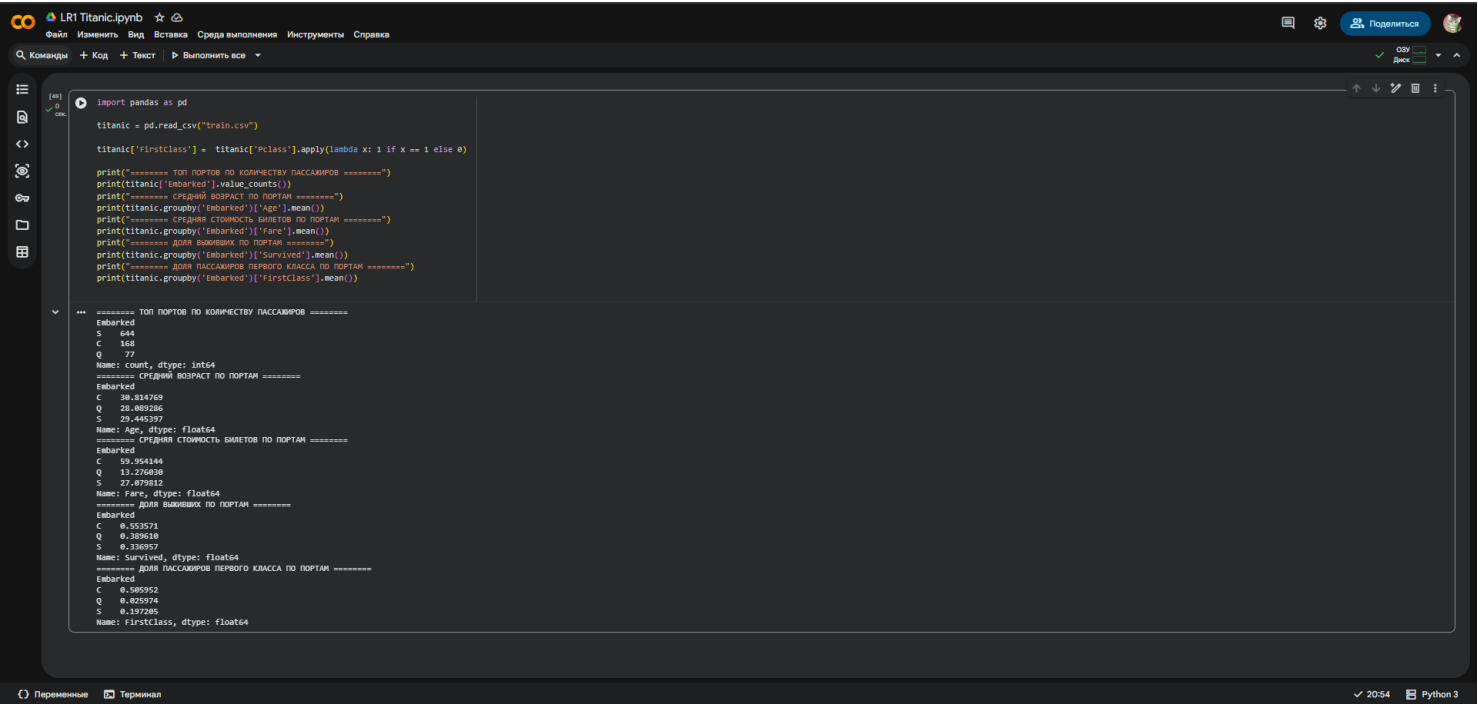


Рисунок 11 – Проанализировал порты посадки.

Анализ: Пассажиры, севшие в порту С (Шербур), выживали чаще всего. Статистика показывает, что из этого порта села наибольшая доля пассажиров самого дорогого, 1-го класса.

Задача 12. Сравнительный анализ по размеру семьи

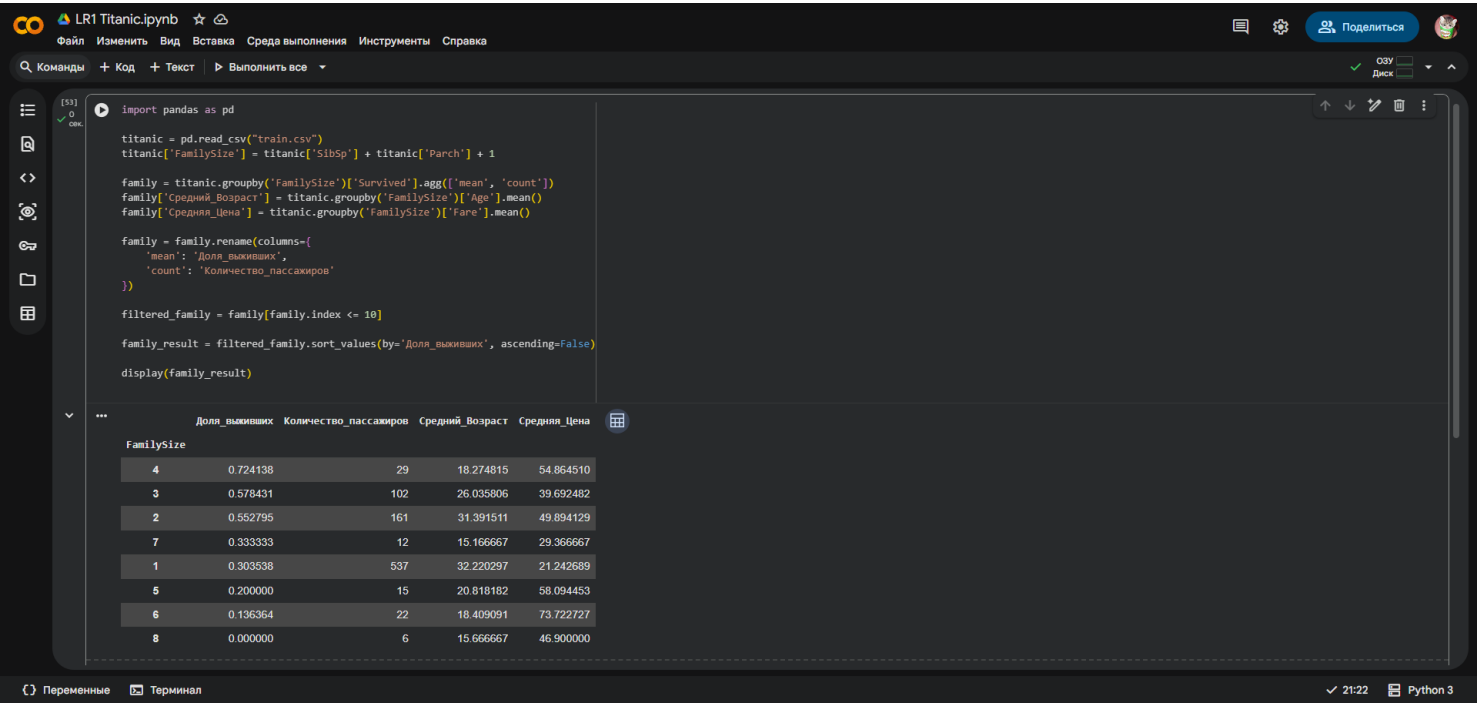


Рисунок 12 – Посчитал выживаемость по размеру семьи.

Сравнение: Максимальная выживаемость (до 72%) была у небольших семей из 2–4 человек. Одиночки и члены больших семей (от 5 человек) погибали значительно чаще.

Ответы на контрольные вопросы

Что такое Pandas и для каких задач используется эта библиотека

Библиотека для анализа и обработки табличных данных в Python. Нужна, чтобы загружать файлы, фильтровать, чистить данные от пропусков и считать статистику.

Чем Series отличается от обычного списка Python и как его создать с пользовательским индексом

В Series каждому элементу жестко присвоен явный текстовый или числовой индекс, а все данные должны быть строго одного типа. Для создания с ручным индексом пишется код `pd.Series(данные, index=[список_индексов])`.

В чём разница между loc и iloc при доступе к элементам Series

Метод `loc` ищет элементы по их текстовым названиям и меткам. Метод `iloc` игнорирует текст и находит элементы только по их системному порядковому номеру от нуля.

Как создать DataFrame из словаря и задать названия колонок и индекс вручную

Для создания нужно передать словарь в конструктор `pd.DataFrame(словарь)`, где ключи автоматически станут колонками. Вручную сетка и имена настраиваются через параметры `columns` и `index` внутри этого же конструктора.

Какие параметры функции `pd.read_csv` вы знаете и для чего используется параметр `usecols`

Основные параметры это путь к файлу, разделитель `sep` и кодировка `encoding`. Параметр `usecols` указывает конкретный список столбцов, которые нужно загрузить, чтобы не забивать память лишними данными.

Какую информацию выводит метод `info` и какие параметры можно ему передать

Выводит общее количество строк, названия колонок, число заполненных ячеек без пропусков, типы данных и объем занятой оперативной памяти. Ему можно передать параметры вроде `verbose` для детальности или `show_counts` для принудительного подсчета непустых ячеек.

Что показывает метод describe для числовых данных и для текстовых данных

Для чисел он выдает количество строк, среднее значение, отклонение, минимум, максимум и проценты. Для текста при указании include object он показывает общее число заполненных строк, количество уникальных текстов, самое частое значение и то, сколько раз оно встретилось.

Как получить доступ к отдельному столбцу DataFrame и какие два способа существуют

Первый способ это запись в квадратных скобках с текстом в кавычках. Второй способ это обращение через точку как к атрибуту, что работает только для простых названий без пробелов.

Как отфильтровать DataFrame по одному условию и как комбинировать несколько условий

Для одного условия логическое выражение пишется внутри квадратных скобок датафрейма. Для комбинации нескольких условий используются знаки амперсанда для логического И или вертикальной черты для логического ИЛИ, при этом каждое условие обязательно оборачивается в круглые скобки.

Какие статистические методы можно применять к DataFrame и Series и что произойдет при использовании mean ко всей таблице с текстом

Можно применять методы расчета среднего mean, медианы median, минимума min, максимума max, суммы sum и стандартного отклонения std. Если вызвать mean на таблицу с текстом, современные версии Pandas выдадут ошибку, поэтому нужно добавлять параметр numeric_only True.

Для чего используется метод groupby и как применить несколько агрегирующих функций одновременно

Метод группирует строки таблицы по одинаковым значениям в выбранной колонке. Чтобы посчитать сразу несколько разных метрик, после группировки вызывают метод agg и передают в него список функций в квадратных скобках.

Чем сводная таблица `pivot_table` отличается от обычной группировки и когда удобнее использовать каждую из них

Группировка `groupby` выдает результат в виде обычных строк. Сводная таблица `pivot_table` строит двумерную сетку, распределяя данные одновременно и по строкам, и по столбцам. Группировка удобна для простых линейных подсчетов, а сводная таблица незаменима для поиска взаимосвязи между двумя признаками сразу.

Как обнаружить и обработать пропущенные значения в `DataFrame` и какие стратегии заполнения пропусков вы знаете

Обнаружить пустые места можно связкой методов `isnull` и `sum`. Стратегии обработки включают в себя полное удаление строк с пропусками через `dropna`, заполнение фиксированным текстом или заполнение рассчитанной статистикой вроде среднего или медианы через метод `fillna`.

Как создать новый признак на основе существующих и для чего используется метод `pd.cut`

Новый признак создается простым математическим расчетом между колонками или применением функции через `apply`. Метод `pd.cut` нужен, чтобы разбить непрерывные числа на диапазоны и превратить их в текстовые категории, например перевести возраст в группы детей и взрослых.

Как отсортировать `DataFrame` по значениям одного или нескольких столбцов и как вывести только топ-N строк

Сортировка выполняется методом `sort_values`, где в параметре `by` указывается имя столбца или список столбцов, а направление задается через `ascending`. Чтобы оставить только топ-N строк, сразу после сортировки дописывается метод `head` с нужным числом внутри скобок.

Вывод

В ходе лабораторной работы я научился работать с библиотекой Pandas на примере датасета «Титаник». Я освоил загрузку файлов, фильтрацию строк по условиям, расчет статистик и группировку данных. Также я научился заполнять пропуски, создавать новые признаки и строить сводные таблицы. В результате я получил практические навыки работы с данными с помощью библиотеки Pandas.